

Professional Vue.js



Completed source code for all labs (for checking your work) can be found at:

<https://github.com/watzthisco/pro-vue>

Version 2.1, 2026
by Chris Minnick
Copyright 2026, WatzThis?
www.watzthis.com



Table of Contents

TABLE OF CONTENTS	2
DISCLAIMERS AND COPYRIGHT STATEMENT	4
DISCLAIMER	4
THIRD-PARTY INFORMATION	4
COPYRIGHT	4
HELP US IMPROVE OUR COURSEWARE	4
CREDITS	5
ABOUT THE AUTHOR	5
SETUP INSTRUCTIONS	6
COURSE REQUIREMENTS	6
CLASSROOM SETUP	6
TESTING THE SETUP	6
INTRODUCTION AND GIT REPO INFO	7
LAB 01 - INSTALLING AND CONFIGURING VS CODE	8
PART 1 - INSTALLING VISUAL STUDIO CODE	8
PART 2: INSTALLING EXTENSIONS	8
PART 3: CREATING A NEW PROJECT	8
LAB 02: GET STARTED WITH VITE	8
INTRODUCING OUR REAL-WORLD PROJECT: CONDUIT	10
LAB 03: YOUR FIRST COMPONENT	11
LAB 04: CREATE MORE COMPONENTS	15
LAB 05: TESTING VUE	18
LAB 06 - GETTING STARTED WITH NODE.JS AND NPM	20
PART 1: GETTING TO KNOW NODE.JS	20
PART 2: USING NPM	22
LAB 07 - INITIALIZE NPM	23
PART 2: HOW TO WRITE AND RUN NPM SCRIPTS	23
LAB 08: MAKING A WEB SERVER	25
LAB 09 - AUTOMATE LINTING	26
LAB 10: MANUAL IN-BROWSER TESTING AND DEBUGGING	28
LAB 11: STATIC VERSION	30
LAB 12: STYLING VUE COMPONENTS	33
LAB 13: COMPUTED PROPERTIES	35
LAB 14: METHODS AND STATE	36
LAB 15: EVENTS	37
LAB 16: COMPONENT LIFECYCLE	38
LAB 17: FORMS	41

LAB 18: SLOTS.....	42
LAB 19: COMPOSABLES	46
LAB 20: IMPLEMENTING PINIA.....	47
LAB 21: ROUTING	51
LAB 22: AJAX	55
LAB 23: TESTING WITH VITEST	57
LAB 24: TRANSITIONS AND ANIMATIONS	59

Disclaimers and Copyright Statement

Disclaimer

WatzThis? takes care to ensure the accuracy and quality of this courseware and related courseware files. We cannot guarantee the accuracy of these materials. The courseware and related files are provided without any warranty whatsoever, including but not limited to implied warranties of merchantability or fitness for a particular purpose. Use of screenshots, product names and icons in the courseware are for editorial purposes only. No such use should be construed to imply sponsorship or endorsement of the courseware, nor any affiliation of such entity with WatzThis?.

Third-Party Information

This courseware contains links to third-party web sites that are not under our control and we are not responsible for the content of any linked sites. If you access a third-party web site mentioned in this courseware, then you do so at your own risk. We provide these links only as a convenience, and the inclusion of the link does not imply that we endorse or accept responsibility for the content on those third-party web sites. Information in this courseware may change without notice and does not represent a commitment on the part of the authors and publishers.

Copyright

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior expressed permission of the owners, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law. For permission requests, write to the owners, at info@watzthis.com.

Help us improve our courseware

Please send your comments and suggestions via email to info@watzthis.com

Credits

About the Author

Chris Minnick is a prolific published author, trainer, web developer and founder of WatzThis, Inc. Minnick has overseen the development of hundreds of web and mobile projects for customers from small businesses to some of the world's largest companies, including Microsoft, United Business Media, Penton Publishing, and Stanford University.

Since 2001, Minnick has trained thousands of Web and mobile developers. In addition to his in-person courses, Chris has written and produced online courses for Ed2Go.com, Skillshare, O'Reilly Media, and Pluralsight.

Minnick has authored and co-authored books and articles on a wide range of Internet-related topics including JavaScript, HTML, CSS, mobile apps, e-commerce, Web design, SEO, and security. His published books include Claude Code For Dummies, Coding with AI For Dummies, A Developer's Guide to Integrating Generative AI into Applications, JavaScript All-in-One For Dummies, ReactJS Foundations, JavaScript for Kids, Writing Computer Code, Coding with JavaScript For Dummies, Beginning HTML5 and CSS3 For Dummies, Webkit For Dummies, CIW eCommerce Certification Bible, and XHTML.

Setup Instructions

Course Requirements

To complete the labs in this course, you will need:

- A computer running macOS, Windows, or Linux.
- Access to the Internet.
- A modern web browser.
- Ability to install software globally (or certain packages pre-installed as specified below).

Classroom Setup

These steps must be completed in advance if the students will not have administrative access to the computers in the classroom. Otherwise, these steps can be completed during the course as needed.

- ☐ 1. Install Node.js on each student's computer.

Go to nodejs.org and download the current LTS release. The labs in this course require Node.js 20.19 or later, or 22.12 or later — Vite 8 will refuse to start on anything older.

- ☐ 2. Install a code editor.

We recommend the free Visual Studio Code editor, which you can download from <https://code.visualstudio.com/>

- ☐ 3. Make sure a Chromium-based browser (Google Chrome or Microsoft Edge) is installed, so the Vue DevTools extension is available.
- ☐ 4. Install git on each student's computer.

If you're using macOS, you already have git installed. If you're on Windows, git can be downloaded from <https://git-scm.com>. Select all the default options during installation.

Testing the Setup

- ☐ 5. Open a command prompt.

Use Terminal on macOS (/Applications/Utilities/Terminal).

Use Git Bash, Windows Terminal, or PowerShell on Windows.

- ☐ 6. Check your Node.js version. It must be 20.19+ or 22.12+.

```
node -v
```

- ☐ 7. Enter `cd` to navigate to the user's home directory (or change to a directory where student files should be created).

- ☐ 8. Enter the following:

```
git clone https://github.com/watzthisco/pro-vue
```

The lab solution files for the course will download into a new directory called `pro-vue`.

- ☐ 9. Enter `cd pro-vue/setup-test` to switch to the test project.
- ☐ 10. Enter `npm install`

This step will take some time. If it fails, the likely problem is that your firewall is blocking access to github.com and/or registry.npmjs.org.

- ☐ 11. When everything is done, enter `npm run dev`
Vite prints a Local URL (`http://localhost:5173/` by default). Open it in your browser — you should see the starter page.
- ☐ 12. Enter `npm run test` to confirm that the test runner works as well.
- ☐ 13. If you get an error, delete the `node_modules` folder (by entering `rm -r node_modules`) and run `npm install` again, followed by `npm run dev`

Introduction and Git Repo Info

Most of the labs in this course build on the labs that came before. So, if you don't complete a lab or can't get a certain lab to work, it's possible that you can get stuck and won't be able to move forward until the error is corrected.

To help you check your work and to make it possible to come into the class at any point, the git repository for this course contains finished versions of every lab.

The url for the course repository is:

`https://github.com/watzthisco/pro-vue`

You can find the finished code for each lab inside the **solutions** directory.

Lab 01 - Installing and Configuring VS Code

Visual Studio Code is an integrated development environment for JavaScript. You can use any code editor or IDE you like, but Visual Studio Code is open source and has good support for modern frameworks like Vue.js and Node.js, as well as built-in integrations with the tools we'll be using in this course and extensions for doing anything else you'll need to do.

None of the labs in this course (other than this one) will depend on any particular IDE; so, if you prefer another editor, feel free to use it and to adapt the instructions in this lab to your own editor.

Part 1 - Installing Visual Studio Code

- ☐ 1. Go to <https://code.visualstudio.com/> .
- ☐ 2. Click the download link.

When the download completes, launch the installer and follow the prompts to install Visual Studio Code.

Part 2: Installing Extensions

VS Code has thousands of available extensions. To make working with Vue.js and related JavaScript tools easier, we'll install a couple of them now.

- ☐ 3. Click the Extensions icon on the left sidebar.
- ☐ 4. Search for and install the Vue - Official extension, by the Vue core team. This gives you Vue syntax highlighting, formatting, and type checking for single-file components.

Note: Vue - Official replaces Vetur, which was the Vue 2 extension and is now deprecated. If you have Vetur installed, disable it — running both at once causes duplicate errors.

- ☐ 5. Search for and install the ESLint extension, by Microsoft, so lint errors appear inline as you type.

Part 3: Creating a New Project

- ☐ 6. Select File > Open Folder, or click the Open Folder link on the start screen.
- ☐ 7. Select the folder where you want to create your project (such as in Documents) and click New Folder to create a new folder. Name the new folder vue-training.
- ☐ 8. Select the new folder and click Open.

Lab 02: Get Started with Vite

Vue projects are scaffolded with create-vue, which builds a project on top of Vite. Vite serves your source files as native ES modules during development, so there is no bundling step to wait for when you save a file.

Note: Vue CLI, which earlier versions of this course used, reached end of life at the end of 2023. create-vue is the tool the Vue documentation now points to.

- ☐ 1. Start a new project in VS Code (if you didn't already do this in the previous lab).
- ☐ 2. Open the terminal (ctrl - `) and scaffold your first app. There is nothing to install globally — npm create downloads the scaffolding tool and runs it.

```
npm create vue@latest conduit
```

If this produces an error, you most likely need to upgrade the version of Node.js on your computer (see the setup instructions).

- ☐ 3. Answer the prompts. For this course, answer No to every optional feature except Vitest, Linter, and Prettier:

TypeScript?	No
JSX Support?	No
Vue Router?	No
Pinia?	No
Vitest for Unit Testing?	Yes
End-to-End Testing?	No
Linter?	Yes
Prettier?	Yes

We add Vue Router and Pinia by hand later in the course, so you can see exactly what each one contributes.

- ☐ 4. If the installer asks you about installing experimental features, don't.
- ☐ 5. Answer 'no' to skipping example code.
- ☐ 6. Go into the new directory and install the dependencies.

```
cd conduit  
npm install
```

- ☐ 7. Test that everything was installed and works.

```
npm run dev
```

- ☐ 8. When the server is ready, Vite prints a Local URL. Open it in a web browser (http://localhost:5173/ by default).

If the app was successfully created, you should see the create-vue welcome page.

- ☐ 9. In your code editor, find App.vue inside the src directory and open it for editing.
- ☐ 10. Change some of the text in the <template> section.
- ☐ 11. Return to your web browser and notice that the page updates without a reload. This is hot module replacement.

If it doesn't update, return to your terminal and check that npm run dev is still running.

- ☐ 12. Have a look at the project Vite created. Two things differ from the Vue CLI layout you may have seen before:

index.html is in the project root, not in public/. It is the entry point of the app, and it loads /src/main.js with a `<script type="module">` tag.

vite.config.js replaces vue.config.js. It is a plain ES module that exports a configuration object.

Introducing Our Real-World Project: Conduit

In this class, we'll be building a real-world application based on **Conduit**, an open source Medium.com clone. This application will make use of a wide variety of Vue.js features and related technologies, including components, authentication, routing, state management, a RESTful API and more.

Before continuing to the next lab, visit an instance of Conduit at <https://vue3-realworld-example-app-mutoe.vercel.app>.

conduit

Home Sign in Sign up

conduit

A place to share your knowledge.

Global Feed



johndoe
Wed Sep 30 2026

♥ 2

How to Learn JavaScript Efficiently

A comprehensive guide to mastering JavaScript from beginner to advanced level

Read more...

beginners javascript programming webdev



janesmith
Wed Sep 30 2026

♥ 2

React Hooks: Best Practices and Common Pitfalls

Essential patterns and anti-patterns when working with React Hooks

Read more...

frontend hooks javascript react



mikewilson
Wed Sep 30 2026

♥ 1

Building Scalable APIs with Node.js

Architectural patterns and best practices for creating robust backend services

Popular Tags

ai api architecture backend beginners datascience frontend hooks javascript machinelearning nodejs programming python react webdev

Lab 03: Your First Component

Vue.js components let you divide your user interface into independent and reusable pieces. The simplest components just output some HTML, given some input.

In this lab, you'll create a component to hold the contents of the page header.

- 1. Create a new file named `TheHeader.vue` in the `src/components` directory.

This component will hold the navigation and logo for our app. At this point it will just render static HTML. We'll be using the Bootstrap CSS framework to style the navigation as a responsive horizontal navbar.

- 2. Inside `TheHeader.vue` create a script element with the `setup` attribute, and a template element:

```
<script setup>
</script>
```

```
<template>
</template>
```

The `setup` attribute is what makes this a Composition API component. Everything you declare at the top level of `<script setup>` — imports, variables, functions — is available in the template automatically. There is no export default and no components object to maintain.

Note: `<script setup>` is compile-time syntax, not a runtime API. The Vue compiler turns it into a normal component definition. The component's name comes from the file name, which is why `Header.vue` can be used as `<Header />` without registering it.

- 3. Inside the template element, create a `nav` element containing a `div` element, and give the `div` element a class of `container`. The container class will cause Bootstrap to style the contents of the `nav` element as a responsive container.

```
<template>
  <nav>
    <div class="container">
    </div>
  </nav>
</template>
```

- 4. Give the `nav` element the Bootstrap navbar class.

```
<nav class="navbar">
```

- 5. Make a logo link in the container by creating a link and giving it the `navbar-brand` class.

```
<div class="container">
  <a href="/" class="navbar-brand">Conduit</a>
```

```
</div>
```

- ☐ 6. Below the logo link create a list using a `ul` element and several `li` elements.

```
<ul>
  <li>Home</li>
  <li>Sign In</li>
  <li>Sign Up</li>
</ul>
```

- ☐ 7. Add Bootstrap's `nav` and `navbar-nav` classes to the `ul` to format it as a navbar, and add the `flex-row` class to make it a horizontal navbar.

```
<ul class="nav navbar-nav flex-row">
```

- ☐ 8. Add `nav-item` classes to each of the `li` elements, and add `p-2` to add some padding around each item.

```
<li class="nav-item p-2">Home</li>
```

- ☐ 9. Turn the text in each `nav-item` into a link and add the `nav-link` class.

```
<li class="nav-item p-2">
  <a class="nav-link">Home</a>
</li>
```

- ☐ 10. When it's done, `TheHeader.vue` should look like this:

```
<script setup>
// A purely presentational component: no state, no props,
no events.
</script>
```

```
<template>
```

```
<nav class="navbar">
  <div class="container">
    <a href="/" class="navbar-brand">Conduit</a>
    <ul class="nav navbar-nav flex-row">
      <li class="nav-item p-2">
        <a class="nav-link">Home</a>
      </li>
      <li class="nav-item p-2">
        <a class="nav-link">Sign In</a>
      </li>
      <li class="nav-item p-2">
```

```

        <a class="nav-link">Sign Up</a>
      </li>
    </ul>
  </div>
</nav>
</template>

```

- ❑ 11. Open App.vue. If it still contains the create-vue welcome page, delete everything in it and start from this:

```

<script setup>
import HelloWorld from './components/HelloWorld.vue';
</script>

<template>
  <div id="app">
    <HelloWorld msg="Vue.js is fun!" />
  </div>
</template>

```

- ❑ 12. Import TheHeader at the top of the <script setup> block in App.vue.

```
import TheHeader from './components/PageHeader.vue';
```

That import is all that is required. Under the Options API you also had to list the component in a components object; with <script setup> the import itself registers it.

Note: include the .vue file extension in the import. Vite resolves extensionless imports for .js and .mjs only, so './components/Header' will fail where Vue CLI used to accept it.

- ❑ 13. Add <TheHeader /> inside the <template> in App.vue, above the HelloWorld component.
- ❑ 14. Delete the contents of the <style> element in App.vue, if there is one.
- ❑ 15. Delete the CSS import in main.js.
- ❑ 16. Open index.html — it is in the project root, not in public/.
- ❑ 17. In your browser, go to <https://getbootstrap.com/docs/5.3/getting-started/introduction/>. Copy the Bootstrap CSS <link> element and paste it into the head element in index.html.

```

<link

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css
/bootstrap.min.css"

rel="stylesheet"

crossorigin="anonymous"

```

/>

- ☐ 18. Copy the Bootstrap bundle `<script>` tag into `index.html`, right above the `</body>` tag.

Note: Bootstrap 5 dropped its jQuery dependency, so — unlike Bootstrap 4 — there is no jQuery script tag to add.
--

- ☐ 19. Start the app (if it's not already running) with `npm run dev`, and view it in your browser.

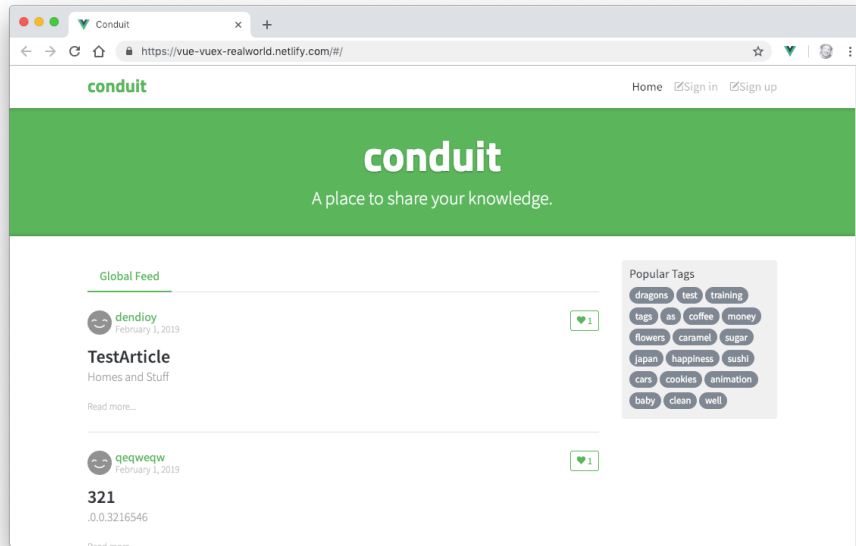
Lab 04: Create More Components

In this lab, you'll create static versions of the main body and the footer for your Vue application.

- ❑ 1. Update the template in App.vue to contain the three main components that will make up the application's Home page: TheHeader, HomeContent, and TheFooter. Remove HelloWorld and TheWelcome from the template and from the imports section in App.vue.

```
<template>
<div id="app">
  <TheHeader />
  <HomeContent />
  <TheFooter />
</div>
</template>
```

- ❑ 2. Make the new .vue files for the HomeContent and TheFooter components (inside src/components) and create <script setup> and <template> sections for each.
- ❑ 3. Put some placeholder text into the new components (for example: <h1>Home</h1> and <h1>Footer</h1>).
- ❑ 4. Import TheFooter and HomeContent into App.vue. With <script setup> the import is the registration — there is no components property to add them to.
- ❑ 5. Run npm run dev and preview your app so far in a browser.
- ❑ 6. Using the following screenshot of the finished Conduit application, write the HTML in the HomeContent.vue component to render a static version of everything underneath the top navigation (without worrying about style yet). The next page contains the finished code, but try to implement it on your own before looking.



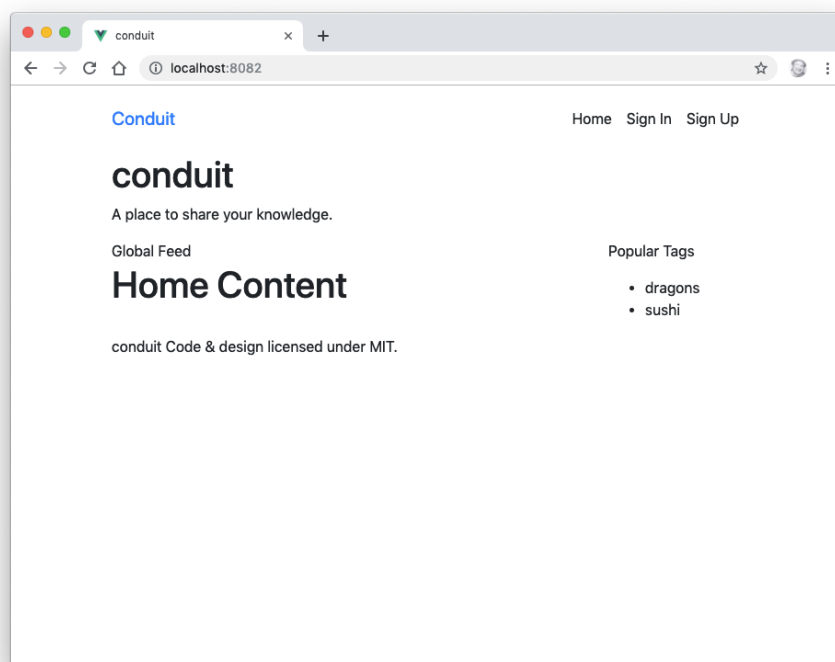
```
<template>
  <div class="home-page">
    <div class="banner">
      <div class="container">
        <h1 class="logo-font">conduit</h1>
        <p>A place to share your knowledge.</p>
      </div>
    </div>
    <div class="container page">
      <div class="row">
        <div class="col-md-9">
          <div class="feed-toggle">
            <ul class="nav nav-pills outline-active">
              <li class="nav-item">Global Feed</li>
            </ul>
          </div>
          <p>Home Content Placeholder</p>
        </div>
        <div class="col-md-3">
          <div class="sidebar">
            <p>Popular Tags</p>
            <div class="tag-list">
              <ul class="tag-list">
                <li class="tag-default tag-pill tag-outline">
                  dragons
                </li>
                <li class="tag-default tag-pill tag-outline">
                  sushi
                </li>
              </ul>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
```

```
</div>
</div>
</template>
```

- 7. Write the HTML for the static version of the TheFooter component.

Here's what the final rendered footer should be:

```
<template>
  <footer>
    <div class="container">
      conduit
      <span class="attribution">
        Code & design licensed under MIT.
      </span>
    </div>
  </footer>
</template>
```



Lab 05: Testing Vue

In this lab, you'll create simple unit tests for each of the components you've made so far. If you answered Yes to the Vitest prompt in Lab 02, the test runner is already installed. Vitest shares Vite's configuration and transform pipeline, so your tests resolve imports and .vue files exactly the way your app does.

- ☐ 1. Make a new directory (at the same level as the src directory) and name it tests.
- ☐ 2. Make a subdirectory of your tests directory and name it components.
- ☐ 3. Make a new file in this directory named Header.spec.js.
- ☐ 4. Import the test functions from vitest, and mount from Vue Test Utils.

```
import { describe, expect, it } from 'vitest';
import { mount } from '@vue/test-utils';
```

Note: this course uses Vue Test Utils 2, which is the version built for Vue 3. Its API differs from version 1 in a few places — most visibly, the propsData mounting option is now called props, and createLocalVue is gone in favour of the global.plugins option.

- ☐ 5. Import the TheHeader component. The @ alias points at src, and is configured in vite.config.js.

```
import TheHeader from '@components/TheHeader.vue';
```

- ☐ 6. Write a function to mount the component.

```
const createWrapper = () => mount(TheHeader);
```

- ☐ 7. Write a describe function to describe your test suite.

```
describe('TheHeader', () => {
  });
```

- ☐ 8. Inside the describe function, create a test.

```
it('should render without a problem', () => {
  const wrapper = createWrapper();

  expect(wrapper.exists()).toBe(true);
});
```

- ☐ 9. Open vite.config.js and confirm it has a test section. If it does not, add one — Vitest needs a DOM implementation to mount components into.

```
test: {
  environment: 'jsdom',
  globals: true,
  include: ['tests/**/*.spec.js'],
},
```

- ☐ 10. If jsdom isn't installed yet, install it.

```
npm install --save-dev jsdom
```

- ☐ 11. Run `npm run test` to check whether it works.
- ☐ 12. Make copies of this spec for the `HomeContent` component and the `TheFooter` component.
- ☐ 13. Modify the contents of the new files to test the new components.
- ☐ 14. Run your tests again and make sure that all of them pass.
- ☐ 15. Try `npm run test:watch`. Vitest re-runs only the tests affected by the file you just saved.

Lab 06 - Getting Started with Node.js and npm

Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine. It can be used to create server-side programs with JavaScript as well as for automating development tasks. In this course, we will be using it for the latter purpose.

Part 1: Getting to Know Node.js

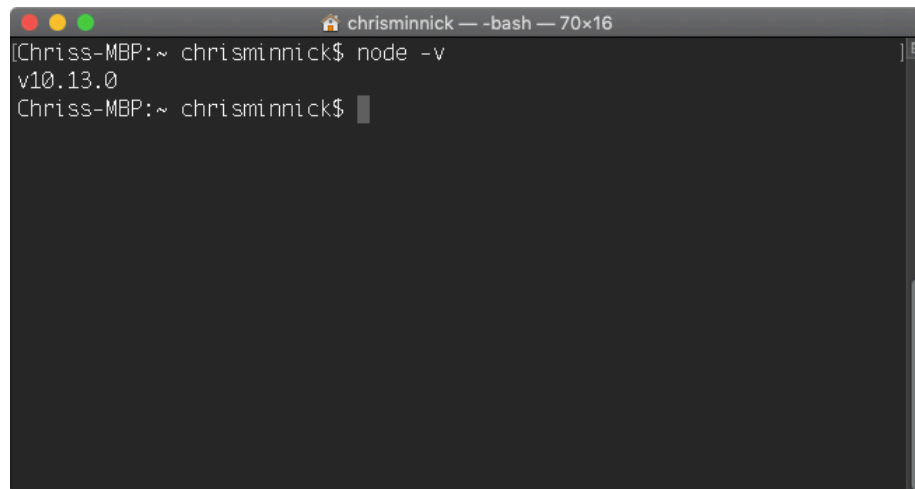
In this part, you will learn the basics of using Node.js.

- ☐ 1. Open a terminal window (such as the one built into VS Code) and change to the labs/lab06 directory.

Note: You can use the `cd` command (MacOS and Windows) to change directories. To go up a directory use `cd ../`

To go into a directory, enter `cd` followed by the name of the directory. You can list the contents of a directory by using `ls` (on MacOS) or `dir` (on Windows).

- ☐ 2. To check whether Node.js is properly installed, enter `node -v`
You should see something like the following:

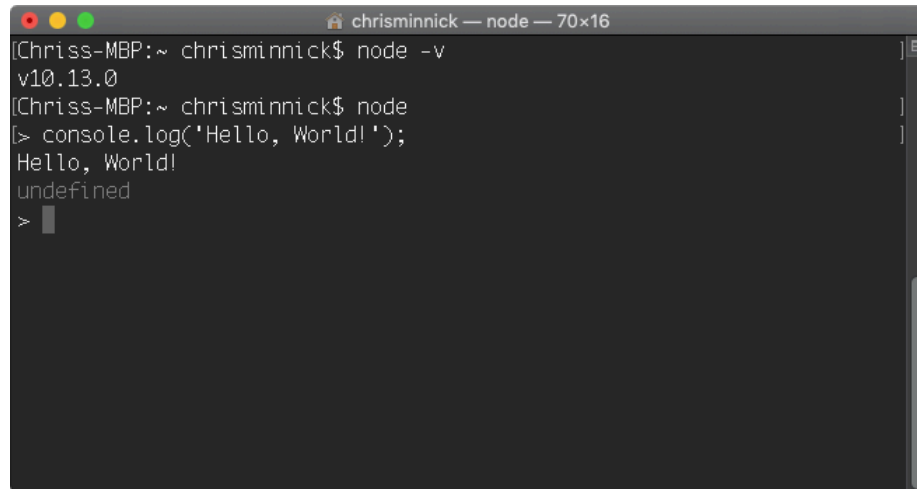
A screenshot of a terminal window with a dark background. The window title is "chrisminnick — -bash — 70x16". The prompt is "Chriss-MBP:~ chrisminnick\$". The command "node -v" has been entered and executed, resulting in the output "v10.13.0". The prompt is now "Chriss-MBP:~ chrisminnick\$ " with a cursor.

```
Chriss-MBP:~ chrisminnick$ node -v
v10.13.0
Chriss-MBP:~ chrisminnick$
```

- ☐ 3. Enter `node` to open the interactive shell, REPL.

Note: You can enter any JavaScript statement into REPL and you have access to all the Node.js modules.

- ☐ 4. Enter `console.log('Hello, World!');` into the shell.

A terminal window titled "chrisminnick — node — 70x16". The prompt is "[Chriss-MBP:~ chrisminnick\$]". The user enters "node -v", and the output is "v10.13.0". The user then enters "node", and the prompt changes to ">". The user enters "console.log('Hello, World!');", and the output is "Hello, World!". The prompt then returns to ">". The user enters "v", and the prompt returns to ">".

```
[Chriss-MBP:~ chrisminnick$ node -v
v10.13.0
[Chriss-MBP:~ chrisminnick$ node
> console.log('Hello, World!');
Hello, World!
undefined
>
v
>
```

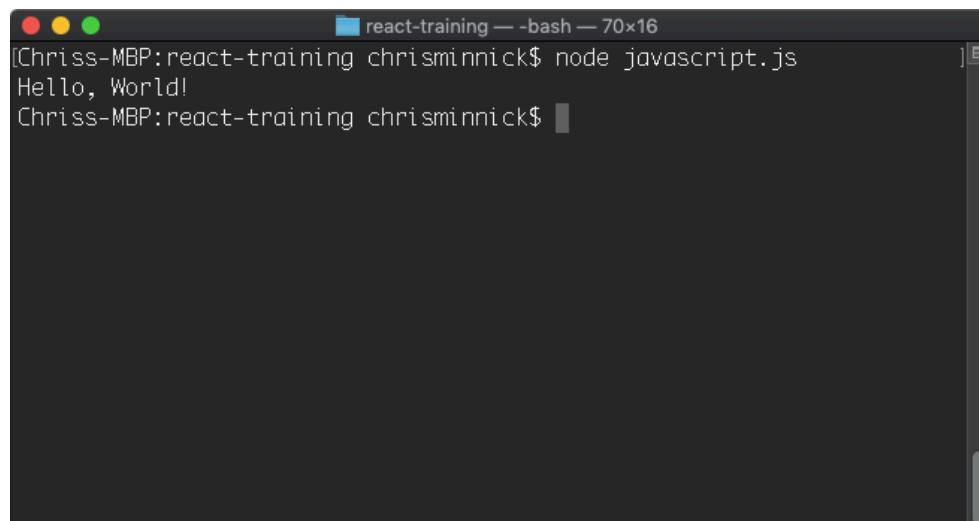
Note: Every JavaScript statement has a return value. The default return value is `undefined`. So, if you execute a command that doesn't have any other return value, as in this case, node outputs `undefined` after the results of running the statement.

You will not normally work with node from the interactive shell. The other way to execute code with node is to write your JavaScript into a file and execute that file.

- ☐ 5. Exit out of REPL by pressing ctrl-D (Mac or Linux) or ctrl-Z (Windows) or by pressing ctrl-C twice.
- ☐ 6. Create a text file using your code editor and enter the following code:

```
console.log('Hello, World!');
```

- ☐ 7. Save the file as **javascript.js**
- ☐ 8. In your terminal, navigate to the directory where you saved javascript.js.
- ☐ 9. Once you've located javascript.js, enter `node javascript.js` to run it.

A terminal window titled "react-training — -bash — 70x16". The prompt is "[Chriss-MBP:react-training chrisminnick\$]". The user enters "node javascript.js", and the output is "Hello, World!". The prompt then returns to "Chriss-MBP:react-training chrisminnick\$".

```
[Chriss-MBP:react-training chrisminnick$ node javascript.js
Hello, World!
Chriss-MBP:react-training chrisminnick$
```

Part 2: Using npm

The node package manager (npm) is the tool for installing and managing node modules created by the node community. In this part, you will learn about the basic npm commands.

- ☐ 1. Enter `npm -v` to find out what version of npm is installed on your computer.
- ☐ 2. Enter `npm ls -g --depth=0`

This command lists the packages that are installed globally on your computer. npm ships with Node.js, so the way to update it is normally to install a newer Node.js release rather than to update npm on its own.

Note: a global npm install writes outside your home directory. If one fails with a permissions error on macOS or Linux, prefer a Node version manager such as nvm or fnm over running npm with sudo.

- ☐ 3. Enter `npm ls` to see the packages installed in the current project only.
- ☐ 4. Enter `npm outdated`

This command compares the versions in your project against the registry and lists anything with a newer release available.

- ☐ 5. Enter `npm help ls`

The help command will show you documentation for a package. On Windows, it may open in a browser. On MacOS, the help will display in the Terminal.

- ☐ 6. If the help file displayed in the console window, type `q` to exit the help system.
- ☐ 7. Enter `npm update` or `npm update -g`

`npm update` will search the npm registry for newer versions of installed packages and install them along with their dependencies.

These are all the basic commands you need to know to get started with npm. In future labs, we will be using npm extensively to install and manage packages and to run tasks.

Lab 07 - Initialize npm

In this lab, you will initialize npm for your project and learn about the package.json file.

- ☐ 1. In your terminal, change to the **labs/lab07** directory and enter:

```
npm init
```

You will be asked some questions to configure npm for your project. The default values will be shown in parentheses after the question. Press Enter or Return to accept each of these default values. Once you have gone through all the questions, you will see that a new file, package.json, has been created in the root of your project.

Note: When using git bash shell on Windows, the configuration script may hang after the last question. When this happens, press Ctrl-C. Everything has run correction and the package.json file has been created, but it just doesn't exit correctly.

- ☐ 2. Open **package.json** in your code editor.

The package.json file configures npm. When you want to install your project in a new directory, you will enter `npm install` and it will follow instructions in this file to do the job.

- ☐ 3. Enter `npm install` in the console.

There's nothing for npm to do at this point, since you don't have any modules installed or instructions inside **package.json**, however a new file named **package-lock.json** will be created in your project.

Part 2: How to Write and Run npm Scripts

In this part, you will learn how to create npm scripts and run them. Npm scripts can be used to automate many of the tasks involved in front-end development, such as testing, building, and deployment.

- ☐ 1. Open package.json in your code editor if it's not already open.

Npm's default package.json file contains a scripts object. If you didn't specify a test script when you ran npm init, a default test method was created for you inside the scripts object.

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1"  
},
```

- ☐ 2. Open your command prompt and enter `npm run`.

Npm's `run` command can be used to run methods inside the `scripts` object. If you use `npm run` without any arguments, it will return a list of the available scripts. In this case, you should get the following:

Lifecycle scripts included in lab-files:

```
test
  echo "Error: no test specified" && exit 1
```

- 3. Enter `npm run test`

The test script will run and output the message saying that no test is specified, and then it will exit with an error.

We'll specify a test in a future lab. For now, we're going to create a simple build script, which will run the test script and then exit with a message.

- 4. Change `exit 1` in the test script to `exit 0` and run the test script again.

`Exit 0` tells node to exit normally without throwing an error.

- 5. Add another property to the `scripts` object, named `build`. For now, the build script will just print out a message.

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "build": "echo \"BUILD OK\" && exit 0"
},
```

- 6. Create a prebuild script. Each script that you create automatically has a pre- and post- script that you can override with your own code. The names of these scripts are the name of your method, with either `pre` or `post` prepended. We'll use the prebuild script to run the test script, and we'll change the test script to output a success message for now. **Make sure to change the exit status to 0 to indicate success.**

```
"scripts": {
  "test": "echo \"All tests passed\" && exit 0",
  "build": "echo \"BUILD OK\" && exit 0",

  "prebuild": "npm run test"
},
```

- 7. Return to the command line, and type `npm run build` at the command line to test your new (very simple) automated build.

Lab 08: Making a Web Server

In this lab, you'll use Node.js and the `http` module to create a simple web server that listens for connections and responds with a simple Hello World message.

- ☐ 1. Run `npm init` inside the labs/lab08 folder. When it asks for the 'type' enter **module** to override the default value (commonjs). This application will use the JavaScript module syntax (`import`) rather than the commonjs syntax (`require`).
- ☐ 2. Open your code editor and create a file named **server.js**
- ☐ 3. Enter the following code inside of `server.js`

```
import http from 'node:http';

const hostname = '127.0.0.1';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at
http://${hostname}:${port}/`);
});
```

Note: the `console.log` statement uses a template literal to combine dynamic values with static text. The characters surrounding the string are backticks (upper left of the keyboard), not single quotes.

Note: `node:` prefixes such as `node:http` identify built-in modules explicitly, so they can never be shadowed by a package of the same name from npm.

- ☐ 4. Because this file uses `import` rather than `require`, Node needs to treat it as an ES module. Open `package.json` and make sure it contains the following:

```
"type": "module"
```

- ☐ 5. Save the `server.js` file.
- ☐ 6. In your terminal, navigate to the labs/lab08 folder and run the program by typing:

```
node server.js
```

If everything works correctly, you should see a message that the server is running.

- ☐ 7. In your web browser, go to the address shown in your console window, which should be **`http://127.0.0.1:3000`**

The server will return a message that will display in your browser.

- ☐ 8. Modify the script to return a different message or a full html page.

Lab 09 - Automate Linting

Linting is a way to perform static code analysis on your files. Static code analysis looks at the syntax (and the style, in some cases) of your JavaScript and alerts you if there are problems.

In this lab, you will install ESLint and use it to check a JavaScript file.

Note: ESLint 9 introduced a new configuration format called flat config. The old `.eslintrc.js` format, and the `--ext` flag that went with it, were removed in ESLint 10. Configuration now lives in a file named `eslint.config.js` that exports an array.

- ☐ 1. If your terminal program isn't already open, open it and go to labs/lab09.
- ☐ 2. Initialize the folder as a node package.

```
npm init
```
- ☐ 3. Install ESLint and the two packages the config file will import.

```
npm install --save-dev eslint @eslint/js globals
```
- ☐ 4. Create a file named `eslint.config.js` in the root of the directory.
- ☐ 5. Import the recommended rule set and the standard global definitions.

```
import js from '@eslint/js';  
import globals from 'globals';
```
- ☐ 6. Export an array. Each object in the array is a configuration that applies to the files it matches; later objects override earlier ones.

```
export default [  
  
];
```
- ☐ 7. Add an `ignores` entry as the first element of the default array. In flat config this replaces the `.eslintignore` file, which ESLint 10 no longer reads.

```
{  
  ignores: ['node_modules/**', 'eslint.config.js'],  
},
```
- ☐ 8. Add ESLint's recommended rules.

```
js.configs.recommended,
```
- ☐ 9. Add a configuration object for your own source files, with the language options and the style rules you want to enforce.

```
{  
  files: ['**/*.js'],  
  languageOptions: {  
    ecmaVersion: 'latest',  
    sourceType: 'module',  
    globals: {  
      ...globals.browser,
```

```

    },
  },
  rules: {
    indent: ['error', 4],
    'linebreak-style': ['error', 'unix'],
    quotes: ['error', 'single'],
    semi: ['error', 'always'],
    'no-console': 'warn',
  },
},

```

Adjust indent, quotes and linebreak-style to match your own preferences. If you're on Windows you may want linebreak-style set to 'windows'.

no-console is set to warn rather than error because we use console.log in later labs.

- ☐ 10. Create a new script in package.json called lint. Note that there is no --ext flag: flat config decides which files to lint through the files property.

```
"lint": "eslint .",
```

I recommend putting it before the "test" script.

- ☐ 11. Run npm run lint.
- ☐ 12. Fix the errors reported by ESLint, or adjust eslint.config.js to fit your desired coding style, until only the no-console warning remains.
- ☐ 13. Make the lint script run prior to the test script in the prebuild script.

Lab 10: Manual In-Browser Testing and Debugging

In this lab, you will get acquainted with your browser's developer tools for inspecting and debugging your front-end code.

- ☐ 1. In your terminal, navigate to the conduit folder you were working with in labs 01-05.
- ☐ 2. Enter `npm run dev` to start the Vite development server.
- ☐ 3. Install the Vue DevTools browser extension from <https://devtools.vuejs.org/getting-started/installation>.

Vite projects can also run the DevTools in-page, without a browser extension, by installing the `vite-plugin-vue-devtools` package. Either approach works for this lab.

- ☐ 4. Restart your browser, then go to `http://localhost:5173`.
- ☐ 5. Press Command-Option-I (on macOS) or Ctrl-Shift-I (Windows) to open the Developer Tools.
- ☐ 6. Dock the Developer Tools to the right side by clicking the Customize button on the right side of the Developer Tools toolbar and selecting Dock Right.
- ☐ 7. Click Elements. The current HTML and CSS of your document (as it exists in the DOM) will appear.
- ☐ 8. Expand the `<div id="app">` element and click on one of the elements inside it.
- ☐ 9. In the styles pane, add `border: 10px solid red;` to the `element.style` object.
- ☐ 10. Right-click an `h1` element and select Hide element.
- ☐ 11. Click the Console tab to open the JavaScript console.

You can also open the JavaScript console at any time by pressing Ctrl-Shift-J (Windows) or Command-Option-J (Mac).

- ☐ 12. Enter the following into the console, followed by Return (or Enter):

```
document.body.innerHTML = '<h1>Here is some new text!</h1>';
```

The content of the document's body element will change to the HTML you just entered.

- ☐ 13. Click the Sources tab.

The JavaScript debugger will open.

- ☐ 14. Vite serves your source files unbundled during development, so you can find `src/App.vue` and `src/components/Header.vue` in the file tree exactly as they are on disk. Open one and click a line number to set a breakpoint.
- ☐ 15. Refresh the page.

Execution of the script will halt prior to the statement running. Examine the different options available and hover over the buttons to find out what they do.

- ☐ 16. Click the Vue tab that the DevTools extension added. Select a component in the tree and inspect its props and state.

Lab 11: Static Version

The first step in creating a Vue UI is to create a static version. In this lab, you'll continue creating the components that will make up the homepage of the Conduit example application, and you'll learn how to pass data from a parent component to child component and to start using directives to make templates dynamic.

The next thing we'll do is to create the list of articles in the center part of the design.

- 1. Make a new component named GlobalFeed. This will contain the list of articles that will display for logged-out users. Use the following:

```
<script setup>
import ArticleList from './ArticleList.vue';
</script>

<template>
  <div class="home-global"><ArticleList type="all" /></div>
</template>
```

The import both brings ArticleList into scope and registers it for the template. There is no components object to maintain.

- 2. Make a component named ArticleList. This will display a list of article previews. Use this template:

```
<template>
  <div>
    <div v-if="articles.length === 0" class="article-
      preview">
      No articles are here... yet.
    </div>
    <ArticlePreview
      v-for="(article, index) in articles"
      :key="article.title + index"
      :article="article"
    />
  </div>
</template>
```

This template uses the v-if directive to check whether there are any articles to display. It then uses v-for to output an ArticlePreview component for each article.

We pass a prop, article, into each ArticlePreview, and a key so Vue can track list items across re-renders.

Note: in Vue 3, :key goes on the element that carries v-for. In Vue 2 you could not use key on a <template v-for>; in Vue 3 that is the required place for it.

- 3. Start the script block by importing the ArticlePreview component (which we'll create in a moment):

```
<script setup>
import ArticlePreview from './ArticlePreview.vue';
</script>
```

- 4. Copy articles.json from the labs/lab11 folder into a subfolder of src named json. This file contains our article data (until we start using a web API in a later lab).

- 5. Import articles.json and ref into ArticleList.

```
import { ref } from 'vue';
import articlesJson from '../json/articles.json';
```

- 6. Declare the component's state. ref() is the Composition API replacement for the data() function: it wraps a value in a reactive container.

```
const articles = ref(articlesJson);
```

Inside <script setup> you read and write a ref through its .value property. In the template, Vue unwraps it for you, which is why the template above says articles.length and not articles.value.length.

- 7. Make a component named ArticlePreview. This will determine how each preview looks and will eventually contain a link to the full article. Use this template:

```
<template>
  <div class="article-preview">
    <h1 v-text="article.title" />
    <p v-text="article.description" />
    <span>Read more...</span>
  </div>
</template>
```

- 8. ArticlePreview receives a prop, article, from its parent component. Declare it with the defineProps compiler macro.

```
<script setup>
defineProps({
  article: { type: Object, required: true },
});
</script>
```

Note: defineProps, defineEmits and defineExpose are compiler macros. They are not imported from anywhere and they only work at the top level of <script setup>.

- ☐ 9. Import the GlobalFeed component into the HomeContent component, and replace the `<p>Home Content Placeholder</p>` in its template with `<GlobalFeed />`.
- ☐ 10. Run `npm run dev` and preview your application in your browser.
- ☐ 11. Create test suites for your new components.

Lab 12: Styling Vue Components

In this lab, you'll learn to style components using Vue.js's scoped CSS.

- ☐ 1. If it's not already running, start up your development server by running `npm run dev`. Open your browser and go to `http://localhost:5173`.
- ☐ 2. In VSCode, open the `HomeContent` component.
- ☐ 3. Create a scoped `<style>` element under the `<script>` element.
- ☐ 4. Create the following CSS selectors inside `<style scoped>`

```
.banner {}  
.banner p {}  
.banner h1 {}  
.feed-toggle {}  
.sidebar {}  
.sidebar p {}
```

- ☐ 5. Add a background color to the `.banner` with a value of `#5cb85c`

```
.banner {  
  background: #5cb85c;  
}
```

- ☐ 6. Center the paragraph text, make it larger and white, and set the font-weight to a lighter value.

```
.banner p {  
  color: #fff;  
  text-align: center;  
  font-size: 1.5rem;  
  font-weight: 300;  
}
```

- ☐ 7. Make the `h1` in the banner bolder, centered and larger.

```
.banner h1 {  
  font-weight: 700;  
  text-align: center;  
  font-size: 3.5rem;  
}
```

- ☐ 8. Give the sidebar a background color, padding, and rounded corners.

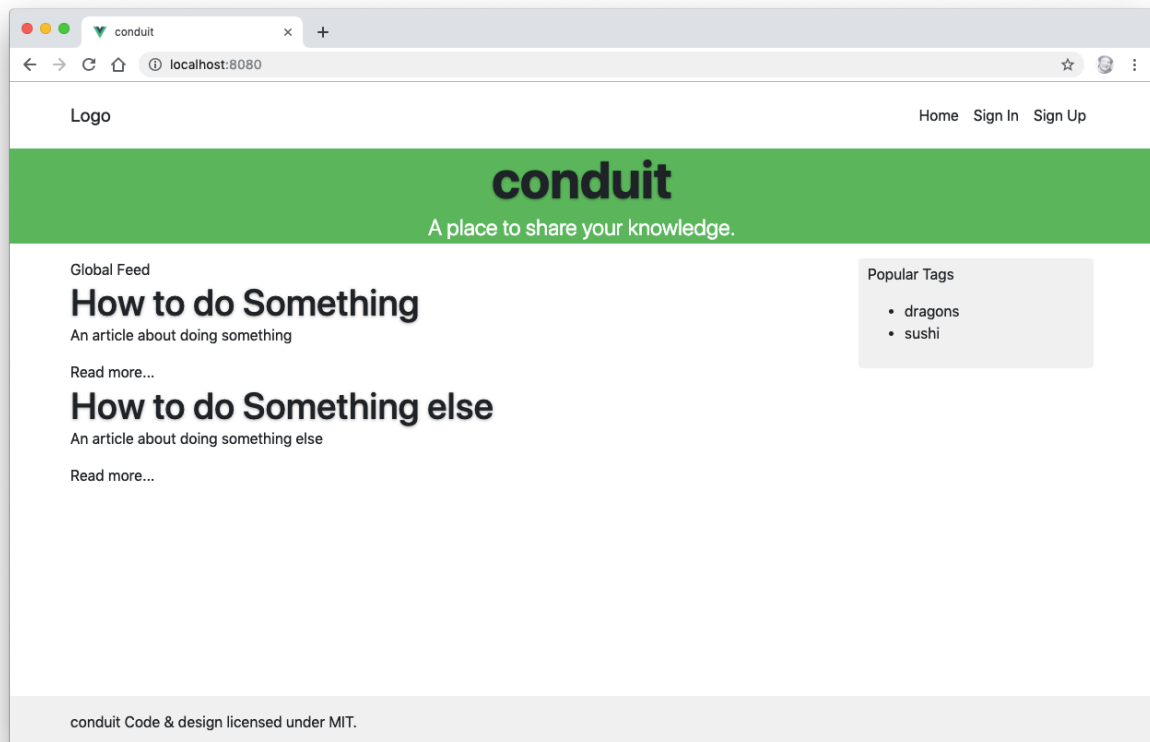
```
.sidebar {  
  padding: 5px 10px 10px;  
  background: #f3f3f3;
```

```
border-radius: 4px;  
}
```

- ☐ 9. Open the TheFooter component.
- ☐ 10. Add a scoped style block to it.
- ☐ 11. Position and style the footer with the following CSS:

```
footer {  
  background: #f3f3f3;  
  margin-top: 3rem;  
  padding: 1rem 0;  
  position: absolute;  
  bottom: 0;  
  width: 100%;  
}
```

- ☐ 12. When it's done, it will look like this:



Lab 13: Computed Properties

In this lab, you'll use a computed property to add links to each article's preview.

- ☐ 1. Open the ArticlePreview component in your code editor.
- ☐ 2. Import computed from vue, and capture the return value of defineProps so you can read the props from JavaScript.

```
import { computed } from 'vue';
```

```
const props = defineProps({  
  article: { type: Object, required: true },  
});
```

In the template you write article directly. In the script block you have to go through the object defineProps returns, hence props.article.

- ☐ 3. Create a computed property called articleLink, which returns the "slug" (or path) to the current article.

```
const articleLink = computed(() => ({ slug:  
  props.article.slug }));
```

computed() returns a read-only ref. Like a computed option in the Options API it is cached, and only re-evaluates when one of the reactive values it read has changed.

- ☐ 4. Use the v-bind shorthand in the ArticlePreview template to link the "Read more..." text to the article slug.

```
<span>  
  <a :href="`/article/${articleLink.slug}`">Read more...</a>  
</span>
```

- ☐ 5. Challenge: add a second computed property that formats the article's createdAt date for display.

Lab 14: Methods and State

In this lab, you'll create a "favorite" button associated with each article preview, which can be toggled on and off. The state of the button is held in a ref, and it is changed by a plain function.

- 1. Import ref into ArticlePreview and declare a boolean named favorited.

```
import { ref } from 'vue';
```

```
const favorited = ref(false);
```

A ref declared inside a component is created fresh for every instance of that component, so each article preview gets its own favorited value. This is what the `data()` function guaranteed under the Options API.

- 2. Create a function inside ArticlePreview named `toggleFavorite()` that flips the value of `favorited`. Remember the `.value`.

```
function toggleFavorite() {  
  favorited.value = !favorited.value;  
}
```

There is no methods object. Any function declared at the top level of `<script setup>` is available to the template.

- 3. Make a button inside the ArticlePreview template that calls `toggleFavorite()` when it's clicked and updates its colour, text, or icon based on the value of `favorited`.

```
<button  
  class="btn btn-sm float-end"  
  :class="{  
    'btn-primary': favorited,  
    'btn-outline-primary': !favorited,  
  }"  
  @click="toggleFavorite"  
>  
  <i class="ion-heart"></i>  
</button>
```

Note: `float-end` is the Bootstrap 5 name for what Bootstrap 4 called `float-right`. The same rename applies to `text-start` / `text-end`.

- 4. Challenge: add a `favoritesCount` ref and display it on the button, so it tracks how many times the button for an article has been clicked.

Lab 15: Events

In this lab, you'll add a font-size control to the ArticleList component that controls the font size of all the article preview instances.

- ❑ 1. Add a ref to the ArticleList component called articleFontSize with a value of 1.

```
const articleFontSize = ref(1);
```
- ❑ 2. Pass the value of articleFontSize into the ArticlePreview component by binding the style attribute:

```
<ArticlePreview  
  :style="{ fontSize: articleFontSize + 'em' }"  
  ...  
>
```
- ❑ 3. Start the development server (npm run dev) and try changing the initial value to 3. You should see the article text get larger.
- ❑ 4. Set articleFontSize back to 1.
- ❑ 5. In ArticlePreview, declare the events this component can raise using the defineEmits macro.

```
const emit = defineEmits(['enlarge-text']);
```

Note: declaring events matters more in Vue 3 than it did in Vue 2. Any listener that is not declared here falls through to the component's root element as a native DOM attribute, which is rarely what you want for a custom event.

- ❑ 6. Create a button inside the ArticlePreview template that emits the enlarge-text event on click.

```
<button @click="emit('enlarge-text', 0.1)">Enlarge  
Text</button>
```

In the Options API you would have written \$emit here. With <script setup> you call the function that defineEmits returned.
- ❑ 7. Listen for the enlarge-text event in the ArticleList component and increase the value of articleFontSize by 0.1 when it's received.

```
<ArticlePreview  
  :style="{ fontSize: articleFontSize + 'em' }"  
  @enlarge-text="articleFontSize += 0.1"  
>
```
- ❑ 8. Add another button named "Shrink Text" and make it work.
- ❑ 9. Challenge: make clicking the button only affect the article description, not the text of the button itself.

Lab 16: Component Lifecycle

In this lab, you'll load the articles from a web API when the component is mounted.

- ☐ 1. Create a folder inside src named common and create a file in it named config.js
- ☐ 2. Inside config.js, export a const named API_URL containing the URL of the public API we'll be replacing our articles.json file with. Reading it from an environment variable lets a class point at a different server without editing code.

```
export const API_URL =  
  import.meta.env.VITE_API_URL ??  
  'https://api.realworld.show/api';
```

Note: Vite exposes environment variables on import.meta.env, and only those whose names begin with VITE_. Create a .env.local file in the project root to override it. process.env is not available in browser code.

- ☐ 3. Export API_URL as the default export for config.js.

```
export default API_URL;
```

- ☐ 4. Install axios.

```
npm install axios
```

Note: earlier versions of this course also installed vue-axios, a plugin that hung axios off the global Vue object as Vue.axios. Vue 3 has no global Vue object — each app created with createApp() is isolated — so instead we create a configured axios instance and import it wherever it is needed. That is a plain ES module, which is easier to test and to reason about.

- ☐ 5. Create a file in src/common named api.service.js and import axios and API_URL into it.

```
import axios from 'axios';  
  
import { API_URL } from '@common/config';
```

- ☐ 6. Create the axios instance that every request will go through.

```
const client = axios.create({ baseURL: API_URL });
```

- ☐ 7. Create a const named ApiService, holding an object with an init() method.

```
const ApiService = {  
  init() {  
    client.defaults.baseURL = API_URL;  
  },  
};
```

- ☐ 8. Export ApiService.

```
export default ApiService;
```

- ☐ 9. Import ApiService into main.js and call init() before mounting the app.

```
import ApiService from '../common/api.service';
```

```
ApiService.init();
```

```
createApp(App).mount('#app');
```

- 10. Add a query method to the ApiService object in api.service.js

```
query(resource, params) {  
  return client.get(resource, params).catch((error) => {  
    throw new Error(`[RWV] ApiService ${error}`);  
  });  
},
```

- 11. Import ApiService and onMounted into ArticleList.vue

```
import { onMounted, ref } from 'vue';
```

```
import ApiService from '../common/api.service';
```

- 12. Write a fetchArticles function that uses the query method.

```
async function fetchArticles() {  
  const { data } = await ApiService.query('articles');  
  
  return data.articles;  
}
```

- 13. Call fetchArticles() from the onMounted lifecycle hook and assign the result to the articles ref.

```
onMounted(async () => {  
  articles.value = await fetchArticles();  
});
```

onMounted is the Composition API equivalent of the mounted option. The hooks are all imported from vue and registered by calling them inside <script setup>: onBeforeMount, onMounted, onBeforeUpdate, onUpdated, onBeforeUnmount and onUnmounted.

Note: Vue 2's beforeDestroy and destroyed hooks were renamed to onBeforeUnmount and onUnmounted in Vue 3. There is no onCreated hook — the body of <script setup> is the created hook.

- 14. Remove the articles.json import from ArticleList.vue and initialise articles to an empty array.

```
const articles = ref([]);
```

Start it as an array, not an object: the template calls `articles.length`, which is undefined on a plain object and would make the "No articles..." message never appear.

- ☐ 15. Make sure the development server is running and refresh your page. The articles should now come from the API.
- ☐ 16. Challenge: fix the footer so that it sticks to the bottom of the list of articles when the list is populated.

Lab 17: Forms

In this lab, you'll implement an input box that filters the currently displayed list of articles based on the text entered into it.

- 1. Create an input in the ArticleList component and bind it with v-model.

```
<input
  v-model="searchDetails"
  class="form-control"
  placeholder="filter articles"
/>
```

Note: the earlier version of this lab used v-model.number. That modifier casts the input to a number, which is wrong here — we are filtering on text. Leave it off.

- 2. Add a ref named searchDetails to hold the search text.

```
const searchDetails = ref('');
```

- 3. Import computed and make a computed property in ArticleList called filterIt.

```
const filterIt = computed(() =>
  articles.value.filter((article) =>
    article.title.includes(searchDetails.value),
  ),
);
```

Both refs are read with .value inside the computed callback. Because this is an arrow function there is no this to capture, so the "var self = this" trick the Options API version needed is gone.

- 4. Modify the v-for directive in ArticleList.vue to use filterIt instead of articles:

```
v-for="(article, index) in filterIt"
```

- 5. Start up your development server and try typing letters into the search form to see how the list is filtered.
- 6. Challenge: make the search case-insensitive.
- 7. Challenge: make the search consider the description text as well as the title.

Lab 18: Slots

In this lab, you'll load the popular tags from the API, display them with slots, show each article's tags, and make clicking a tag filter the article list.

- 1. Create a new component, `src/components/TagItem.vue`, that displays a single tag.

```
<template>
  <li class="tag-default tag-pill tag-outline">{{ tag
}}</li>
</template>
<script setup>
defineProps({
  tag: { type: String, required: true },
})
</script>
```

- 2. Create `src/components/TagList.vue`. It owns the layout (the sidebar box, the heading, and the `<ul>`), but not the content. Use a `<slot>` where the tags go, with fallback text for when the parent supplies nothing.

```
<template>
  <div class="sidebar">
    <p>Popular Tags</p>
    <ul class="tag-list">
      <slot>No tags available.</slot>
    </ul>
  </div>
</template>
```

- 3. In `HomeContent.vue`, import `ApiService`, `TagList`, `Tag`, and `onMounted/ref`. Add a tags ref and load the tags from the `/tags` endpoint (same base URL as the articles feed).

```
const tags = ref([])

onMounted(async () => {
  const { data } = await ApiService.query('tags')
  tags.value = data.tags
})
```

- 4. Replace the hard-coded sidebar markup in `HomeContent.vue` with `TagList`, and supply the tags through its default slot.

```

<TagList>
  <TagItem v-for="tag in tags" :key="tag" :tag="tag" />
</TagList>

```

Reload the page. The tags from the API should appear in the second column.

- 5. In ArticlePreview.vue, add a named slot called tags beneath the “Read more...” link.

```

<slot name="tags" />

```

- 6. In ArticleList.vue, fill the tags slot of each ArticlePreview with the article's tagList. Use the #tags shorthand for v-slot:tags.

```

<ArticlePreview ...>
  <template #tags>
    <ul class="tag-list">
      <li
        v-for="t in article.tagList"
        :key="t"
        class="tag-default tag-pill tag-outline"
      >
        {{ t }}
      </li>
    </ul>
  </template>
</ArticlePreview>

```

- 7. Make TagItem.vue emit an event when clicked. Use defineEmits and emit the tag name.

```

<li class="tag-default tag-pill tag-outline"
  @click="emit('select', tag)">
  {{ tag }}
</li>

const emit = defineEmits(['select'])

```

- 8. In HomeContent.vue, add a selectedTag ref and a handler. Clicking the selected tag a second time clears the filter.

```

const selectedTag = ref('')

function selectTag(tag) {
  selectedTag.value = selectedTag.value === tag ? '' : tag
}

```

Listen for the event on each Tag:

```
<TagItem v-for="tag in tags" :key="tag" :tag="tag"
@select="selectTag" />
```

- 9. Pass selectedTag down to the article list. Add a tag prop to GlobalFeed.vue and ArticleList.vue.

In HomeContent.vue:

```
<GlobalFeed :tag="selectedTag" />
```

In GlobalFeed.vue:

```
defineProps({ tag: { type: String, default: '' } })
```

```
<ArticleList type="all" :tag="tag" />
```

In ArticleList.vue:

```
const props = defineProps({ tag: { type: String, default:
'' } })
```

- 10. Update the filterIt computed property in ArticleList.vue so it also checks the selected tag.

```
const filterIt = computed(() => {
  const term = searchDetails.value.toLowerCase()
  return articles.value.filter(
    (article) =>
      (!props.tag || (article.tagList ??
[]).includes(props.tag)) &&
      (article.title.toLowerCase().includes(term) ||
      (article.description ??
'').toLowerCase().includes(term)),
    )
  })
```

- 11. Start up your development server. Verify that tags load, each article lists its tags, and clicking a tag filters the list.
- 12. Add an active class to TagItem.vue so the selected tag is highlighted. Add an active Boolean prop and pass :active="tag === selectedTag" from HomeContent.vue.
- 13. Challenge: allow more than one tag to be selected at once. Change selectedTag to an array, and show only articles that have all of the selected tags. Add a "Clear filters" button, supplied to TagList through a second named slot called footer.
- 14. Challenge: turn TagList into a scoped-slot component that fetches its own tags. Move the ApiService call from HomeContent.vue into TagList.vue, and expose { tags, loading, error } to the parent with slot props (<slot :tags="tags" :loading="loading" :error="error">). In HomeContent.vue, use v-slot="{ tags,

loading, error }" to render a "Loading..." message, an error message, or the list of Tag components.

Lab 19: Composables

A composable is a function that uses the Composition API to encapsulate and reuse stateful logic. Composables are what Vue 3 offers in place of mixins.

Note: mixins still work in Vue 3, but the documentation marks them as legacy. Their problems are that the source of an inherited property is invisible at the point of use, that name collisions between mixins are resolved silently, and that a mixin cannot take parameters. A composable has none of these problems: you pass its inputs in and you name its outputs on the way out.

- ☐ 1. Make a directory inside src named composables.
- ☐ 2. Create a file in it named useArticleFilter.js

By convention a composable is named use followed by what it does, and lives in its own file.

- ☐ 3. Export a function that takes the list of articles as an argument and returns the search text and the filtered list.

```
import { computed, ref } from 'vue';

export function useArticleFilter(articles) {
  const searchDetails = ref('');

  const filterIt = computed(() =>
    (articles.value ?? []).filter((article) =>
      article.title.includes(searchDetails.value),
    ),
  );

  return { searchDetails, filterIt };
}
```

Note that articles arrives as a ref, so the composable reads it through .value and stays reactive to changes made by the caller.

- ☐ 4. Use the composable in ArticleList.vue, and delete the local searchDetails ref and filterIt computed property.

```
import { useArticleFilter } from
'../composables/useArticleFilter';

const { searchDetails, filterIt } =
useArticleFilter(articles);
```

- ☐ 5. Confirm the filter still works in the browser.
- ☐ 6. Challenge: implement filtering of the tags in the right column, reusing the same composable.

Lab 20: Implementing Pinia

In this lab, you'll convert the Conduit application to use a centralized store with Pinia.

Note: earlier versions of this course used Vuex. Vuex is now in maintenance mode and Pinia is the officially recommended store for Vue. The concepts carry over almost one for one — state and getters keep their names, and actions absorb what Vuex split between actions and mutations. Pinia has no mutations at all: an action assigns to state directly. It also drops the string action-type constants and the dispatch/commit indirection, so a store method is called like an ordinary function and can be type-checked and jumped to in an editor.

- ☐ 1. Install Pinia.

```
npm install pinia
```

- ☐ 2. Import createPinia in main.js and install it on the app.

```
import { createApp } from 'vue';
import { createPinia } from 'pinia';
```

```
const app = createApp(App);
```

```
app.use(createPinia());
```

```
app.mount('#app');
```

Unlike a Vuex store, a Pinia instance holds no state of its own. Each store is defined in its own file and registers itself the first time it is used.

- ☐ 3. Make a new directory in src named stores.
- ☐ 4. Make a new file, home.js, inside stores.
- ☐ 5. Make a new API service in api.service.js called ArticlesService.

```
export const ArticlesService = {
  query(type, params) {
    return ApiService.query(`articles${type === 'feed' ?
'/feed' : ''}`, {
      params,
    });
  },
};
```

- ☐ 6. Import defineStore and ArticlesService into stores/home.js

```
import { defineStore } from 'pinia';
```

```
import { ArticlesService } from '@common/api.service';
```

- 7. Define and export the store. The first argument is a unique id; it is what the Vue DevTools label the store with.

```
export const useHomeStore = defineStore('home', {  
  });
```

- 8. Add the state. As in a component, state is a function that returns a fresh object.

```
state: () => ({  
  tags: [],  
  articles: [],  
  isLoading: true,  
  articlesCount: 0,  
}),
```

The Vuex version of this store also declared getters called `articles`, `isLoading`, `articlesCount` and `tags` that just returned the matching piece of state. In Pinia, state is exposed on the store directly, so those pass-through getters are unnecessary. Keep getters for values you actually derive.

- 9. Add the actions. An action can be async, and assigns to state through this — there is no commit and no mutation to write.

```
actions: {  
  async fetchArticles(params = {}) {  
    this.isLoading = true;  
  
    try {  
      const { data } = await ArticlesService.query(  
        params.type,  
        params.filters,  
      );  
  
      this.articles = data.articles;  
      this.articlesCount = data.articlesCount;  
    } finally {  
      this.isLoading = false;  
    }  
  },  
},
```

The finally block guarantees the loading flag is cleared even if the request fails, which the Vuex version did not do.

- 10. Add two more actions, `setTags` and `updateArticleInList`, replacing the mutations of the same names.

```
setTags(tags) {
  this.tags = tags;
},

updateArticleInList(data) {
  this.articles = this.articles.map((article) =>
    article.slug === data.slug
    ? {
      ...article,
      favorited: data.favorited,
      favoritesCount: data.favoritesCount,
    }
    : article,
  );
},
```

- 11. Import the store and `storeToRefs` into `ArticleList.vue`

```
import { storeToRefs } from 'pinia';

import { useHomeStore } from '../stores/home';
```

- 12. Call the store's composable and pull out the pieces you need.

```
const homeStore = useHomeStore();
const { articles, isLoading } = storeToRefs(homeStore);
```

Note: a Pinia store is a reactive object, so destructuring it directly would break reactivity — you would copy the values out and never see an update. `storeToRefs` converts state and getters into refs that keep tracking. Actions are plain functions and can be destructured normally, or called on the store as we do below.

- 13. Remove the local `articles` ref from `ArticleList.vue` — the store owns that state now.
- 14. Change the `onMounted` hook to call the store action.

```
onMounted(() => {
  homeStore.fetchArticles({ type: 'all' });
});
```

Compare that with the Vuex version, which read `this.$store.dispatch(FETCH_ARTICLES, this.listConfig)` and needed a string constant and a computed property to build the payload.

- 15. Add a v-if directive to the ArticleList template to hide the list until the request has finished.

```
<template>
  <div>
    <div v-if="isLoading" class="article-preview">Loading
articles...</div>
    <div v-else>
      <input
        v-model="searchDetails"
        class="form-control"
        placeholder="filter articles"
      />
      <div v-if="articles.length === 0" class="article-
preview">
        No articles are here... yet.
      </div>
      <ArticlePreview
        v-for="(article, index) in filterIt"
        :key="article.title + index"
        :style="{ fontSize: articleFontSize + 'em' }"
        :article="article"
        @enlarge-text="articleFontSize += 0.1"
      />
    </div>
  </div>
</template>
```

- 16. Start the development server and view the app in a browser.
- 17. Open the Vue DevTools and find the Pinia tab. Watch isLoading flip as the request completes, and try editing articles directly from the inspector.
- 18. Challenge: display tags along with each article preview.
- 19. Challenge: populate the Popular Tags box with a list of tags from the currently displayed article previews.
- 20. Challenge: clicking on a tag in the list of popular tags should filter the display of articles.

Lab 21: Routing

So far, our application only has one screen. In this lab, you'll start to implement additional routes. Routing in modern JavaScript frameworks works by reacting to changes in the browser address to load or show different views. In reality everything is happening in the same HTML page, but the data and view change. This is what we mean by a Single Page Application (or SPA).

In this lab, you'll install and configure Vue Router, and then create the first two additional routes for this application: Sign In and Sign Up.

- ☐ 1. Install vue-router.

```
npm install vue-router
```

- ☐ 2. Make a new directory in `src` named `router`.
- ☐ 3. Create a file named `index.js` inside `src/router`.
- ☐ 4. Import the router factory functions. Vue Router 4 and later are created with a function rather than a constructor, and are not installed with `Vue.use`.

```
import { createRouter, createWebHistory } from 'vue-router'
```

Note: this is the biggest API change from Vue Router 3. `new Router()` becomes `createRouter()`; the `mode: 'history'` option becomes `createWebHistory()`; `mode: 'hash'` becomes `createWebHashHistory()`; and the catch-all path `*` becomes `/:pathMatch(.*)*`.

- ☐ 5. Create and export the router, passing it a history implementation and a routes array.

```
export default createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [],
})
```

`createWebHistory` uses the HTML5 History API, so URLs have no #. That is why the `/#/` you may have seen in older versions of this app is gone.

- 6. Make the first (default) route. Remember the `.vue` extension in the dynamic import.

```
routes: [  
  {  
    name: 'home',  
    path: '/',  
    component: () =>  
      import('@components/HomeContent.vue'),  
  },  
],
```

Writing the component as an arrow function that returns an `import()` makes it a lazily-loaded route: Vite gives that route its own JavaScript chunk, which is not downloaded until the route is visited.

- 7. Import the router module into `main.js` and install it on the app.

```
import router from './router'
```

```
app.use(createPinia())  
app.use(router)  
app.mount('#app')
```

Install Pinia before the router, so that navigation guards and route components can reach the stores.

- 8. Open `App.vue` and replace the `<HomeContent />` component with `<RouterView />`.

```
<script setup>  
import { RouterView } from 'vue-router'  
  
import TheHeader from './components/TheHeader.vue'  
import TheFooter from './components/TheFooter.vue'  
</script>  
  
<template>
```

```

<div id="app">
  <TheHeader />
  <RouterView />
  <TheFooter />
</div>
</template>

```

`app.use(router)` **does register** `<RouterView>` and `<RouterLink>` globally, so the import is optional. Importing them explicitly is worth the one line: the editor can then resolve them, and the component says what it depends on.

- ☐ 9. Remove the import for `HomeContent` from `App.vue`.
- ☐ 10. Run the development server and open the application in your browser. It should work the same as before.
- ☐ 11. Open the `TheHeader` component for editing.
- ☐ 12. Find the Sign In link, and replace it with a `RouterLink` component:

```

<RouterLink class="nav-link" active-class="active" :to="{
  name: 'login' }">
  Sign In
</RouterLink>

```

Note: in Vue Router 4 and later, `active-class` is applied on partial matches and `exact-active-class` on exact ones, so use `exact-active-class` if you need exact matching.

- ☐ 13. View the application in your browser and click on the Sign In link. It won't work yet, because the login route doesn't exist.
- ☐ 14. Create a new route, in `router/index.js`, with the name `login` and the path `/login`.

```

{
  name: 'login',
  path: '/login',
  component: () => import('@components/LoginPage.vue'),
},

```

- ☐ 15. Make a new component named `LoginPage` and put a login form in the template.

```

<script setup>
// The form is wired up to the auth store in the next lab.
function onSubmit() {}
</script>
<template>
  <div>

```

```

<h1>Log In Here</h1>
<form @submit.prevent="onSubmit">
  <fieldset class="form-group">
    <input
      class="form-control form-control-lg"
      type="text"
      placeholder="Email"
    />
  </fieldset>
  <fieldset class="form-group">
    <input
      class="form-control form-control-lg"
      type="password"
      placeholder="Password"
    />
  </fieldset>
  <button class="btn btn-lg btn-primary pull-xs-right">Sign in</button>
</form>
</div>
</template>

```

- 16. Use `<RouterLink>` to link the logo to the home page.

```

<RouterLink class="navbar-brand" :to="{ name: 'home' }">
  Conduit
</RouterLink>

```

- 17. Link the Home navigation link to the home page the same way.
- 18. **Challenge:** make the Sign Up page and route.
- 19. **Challenge:** use a `<RouterView>` inside `HomeContent.vue` to render a different list (or differently-styled list) of articles when the user clicks a link. Hint: add a `children` property to the home route and make it an array of route objects.

Lab 22: AJAX

In this lab, you'll implement sign up functionality along with login and checking of authorization.

- ❑ 1. Copy `jwt.service.js` and `api.service.js` from `labs/lab22/common` into `src/common`, **overwriting your existing `api.service.js`**.

The new `api.service.js` adds the `get`, `post`, `put` and `delete` helpers the auth store needs, plus a `setHeader()` method that attaches the stored token to every subsequent request.

- ❑ 2. Copy `stores/auth.js` from `labs/lab22/stores` into `src/stores`.
- ❑ 3. Open `src/stores/auth.js` and read through it. Note the shape:
 - `state` holds `errors`, `user`, and `isAuthenticated`, which is seeded from whatever token is already in `localStorage`.
 - `setAuth` and `purgeAuth` are ordinary actions. In the Vuex version these were the `SET_AUTH` and `PURGE_AUTH` mutations, referred to through string constants in `mutations.type.js`. Pinia needs neither the constants nor the file.
 - `login`, `register`, `checkAuth` and `updateUser` are `async` actions that call the API and then call `setAuth`. They store the server's validation errors on `state.errors` and re-throw, so the component can decide what to do.
- ❑ 4. Copy `Login.vue` and `Register.vue` from `labs/lab22/components` into `src/components`. The ESLint config requires multi-word component names, so save them as `LoginPage.vue` and `SignUpPage.vue`, overwriting your existing versions.
- ❑ 5. Open `LoginPage.vue` and note how the component talks to the store.

```
const router = useRouter()
const authStore = useAuthStore()
const { errors } = storeToRefs(authStore)
async function onSubmit() {
  try {
    await authStore.login({ email: email.value, password:
password.value })
    router.push({ name: 'home' })
  } catch {
    // Errors are surfaced through the store's `errors`
state.
  }
}
```

Note: inside `<script setup>` there is no `this`, so `this.$router` and `this.$store` are not available. `useRouter()` and the store's own `use` function are the Composition API equivalents. `useRoute()` gives you the current route object.

- ☐ 6. Make sure `router/index.js` has a `register route` pointing at `SignUpPage.vue`.
- ☐ 7. Test it out by registering and then by logging in.
- ☐ 8. Import the auth store and `storeToRefs` into `HomeContent.vue`.

```
import { storeToRefs } from 'pinia'
import { useAuthStore } from '@stores/auth'
const { isAuthenticated } = storeToRefs(useAuthStore())
```

The Vuex version used `...mapGetters(['isAuthenticated'])` inside a `computed` option. `storeToRefs` is the direct replacement, and it works on state as well as getters.

- ☐ 9. Use `isAuthenticated` in `HomeContent.vue` to conditionally show a logged-in user message in the sidebar.

```
<p v-if="isAuthenticated">Welcome, authenticated user</p>
```

Note: the Vue 2 version of these components displayed errors with a filter: `{{ v | error }}`. Filters were removed in Vue 3. Use a method, a computed property, or an inline expression instead; `LoginPage.vue` joins the array of messages inline.

- ☐ 10. **Challenge:** display a `logout` button instead of the `Login` and `Sign Up` buttons when a user is logged in.
- ☐ 11. **Challenge:** call the auth store's `checkAuth()` action when the app starts, so a returning user with a valid token is logged straight back in.

Lab 23: Testing with Vitest

If you run the unit tests in your project now, you'll see that the test on `HomeContent.vue` fails. `HomeContent.vue` now depends on the router and on a Pinia store, and neither exists inside a bare `mount()`. In this lab, you'll learn how to provide them.

- ❑ 1. Run `npm run test` and read the failure carefully. The message names the injection that could not be resolved.
- ❑ 2. Visit <https://test-utils.vuejs.org/guide/> and read about the `global.plugins` mounting option.

Note: Vue Test Utils 1 used `createLocalVue()` to install plugins without polluting the global Vue. Vue 3 apps are already isolated, so `createLocalVue` is gone. Plugins are passed per-mount through `global.plugins`.

- ❑ 3. Open `tests/components/HomeContent.spec.js` and create a router for the test.

```
import { createRouter, createWebHistory } from 'vue-router'
const router = createRouter({
  history: createWebHistory(),
  routes: [{ name: 'home', path: '/', component: {
    template: '<div />' } }],
})
```

- ❑ 4. Create a fresh Pinia instance before each test.

```
import { createPinia, setActivePinia } from 'pinia'
let pinia
beforeEach(() => {
  pinia = createPinia()
  setActivePinia(pinia)
})
```

Creating the Pinia instance per test keeps store state from leaking from one test into the next.

- ❑ 5. Pass both plugins when you mount the component.

```
const createWrapper = () =>
  shallowMount(HomeContent, { global: { plugins: [pinia,
    router] } })
```

`shallowMount` stubs out child components, so `GlobalFeed` never mounts. `HomeContent` itself still requests `/tags` when it mounts, so add a `vi.mock()` for `@/common/api.service` that returns `{ data: { tags: [] } }` and keeps the test off the network.
- ❑ 6. Run `npm run test` again and confirm every test passes.

- 7. **Challenge:** write a test for the auth store itself. A Pinia store can be tested without mounting any component: call `useAuthStore()` after `setActivePinia()` and assert on its state.
- 8. **Challenge:** use `vi.mock()` to fake the `api.service` module so the store test never touches the network.

Lab 24: Transitions and Animations

Vue's built-in `<Transition>` component applies CSS classes at each stage of an element entering or leaving the DOM, so you can animate it with plain CSS.

Note: the class names changed in Vue 3. What Vue 2 called `v-enter` is now `v-enter-from`, and what it called `v-leave` is now `v-leave-from`. (`v-leave-to` is unchanged.) If you copy an example from an old blog post and nothing animates, this is usually why.

- ☐ 1. Add a **Remove From Feed** icon (maybe a trash can) to each article.
- ☐ 2. When clicked, it should remove the article from the feed.
- ☐ 3. Wrap the list in a `<TransitionGroup>` and animate articles as they leave.

```
<TransitionGroup name="article" tag="div">
  <ArticlePreview
    v-for="article in visibleArticles"
    :key="article.slug"
    :article="article"
  />
</TransitionGroup>
```

Every child of a `<TransitionGroup>` needs a stable, unique key. Using the array index will make the animation attach to the wrong element.

- ☐ 4. Write the transition classes.

```
.article-enter-from,
.article-leave-to {
  opacity: 0;
  transform: translateX(30px);
}

.article-enter-active,
.article-leave-active {
  transition: all 0.3s ease;
}
```

- ☐ 5. Challenge: add `.article-move` so the remaining articles slide up smoothly to fill the gap.

Hint: the move transition only works if leaving items are taken out of the layout flow. Give `.article-leave-active` `position: absolute` and a width, and make the `<TransitionGroup>` `container position: relative`.